

CS 378 Project 2 Report

Kaleb Gordon

Overview

This project implements modules of the RSA cryptosystem.

- **math_tools.py:**
 - Contains the math algorithms.
- **key_setup.py:**
 - Computes and outputs the keys, public and private.
- **encryption.py:**
 - Takes the public key and a plaintext message as the inputs, outputs the ciphertext.
- **decryption.py:**
 - Takes the public key, the private key and the ciphertext as the inputs, outputs the plaintext.

Algorithms Implemented

- **Modular exponentiation algorithm:**
 - Used to compute $x^a \pmod n$. This is used in the key setup, encryption, and decryption modules to handle the large numbers.
- **Miller-Rabin Primality Test:**
 - Used to generate large prime numbers. It checks if a large random integer is prime by running a set number of test rounds.
- **Extended Euclid Algorithm (EEA):**
 - Needed to compute multiplicative inverses. I used this to calculate the private key d .

Execution Instructions

1. Run *python3 key_setup.py*
 - This generates two prime numbers with 100 decimal digits each, making sure the difference between them is at least $10^{(95)}$. It saves them to two separate files named `public_key` and `private_key`.
2. The plaintext integer should be in a file named `message`.
3. Run *python3 encryption.py*
 - This encrypts the message using the public key and outputs it to a file named `ciphertext`.
4. Run *python3 decryption.py*
 - This outputs the decrypted message and stores it in a new file named `decrypted_message`.

Testing and Verification

To make sure the code worked I used the single integer of 180 digits provided in the assignment as my plaintext.

1. I ran the key setup to generate my primes.
 2. I ran the encryption script, then the decryption script.
 3. I checked my public_key file to make sure 'n' successfully generated as a 200-digit number.
 4. I opened decrypted_message and verified that it perfectly matched the original 180-digit message.
- All of these test files are included in the test_results subdirectory.

Problems Encountered

- Generating large prime numbers takes a really long time to run on my MacBook. To speed things up while writing the code, I changed the code to generate smaller primes, so I didn't have to wait several minutes just to see if my code worked. I changed it back to 100 digits for the final check.
- Writing the EEA and modular exponentiation algorithms from nothing was hard. Stright math is not my favorite part of computer science, so tracking all the variables and remainders through the while loops took a lot of debugging. A big reason why I made smaller generated numbers.
- I got errors early on if my plaintext message was larger than 'n'. I had to add a check in encryption.py to make sure the message was within the bound given by 'n' before the script tried to encrypt it.